Build Your First Vector Database

A Step-by-Step Chroma & RAG Lab Guide

Using Google Colab · No API Key Required · Free

Session	Session 6 of 12 · Fine-Tuning & RAG
Presenter	Dinesh Wadhwani
Duration	~15 minutes (instructor demo) or ~25 minutes (hands-on)
Tools	Google Colab, ChromaDB, pypdf, sentence-transformers
Document	AI for Teachers: an Open Textbook (CC BY 4.0)

This guide walks you through every click, every code cell, and every teaching moment of the Chroma vector database lab. Each step includes exact instructions and an explanation of what is happening technically so you can narrate it live to your students.

WHAT THIS LAB DEMONSTRATES

Lab Overview & Learning Objectives

Students will follow along as you live-demonstrate the complete RAG retrieval pipeline from scratch, using a real academic textbook as the knowledge base. By the end, students will have seen exactly how a production RAG system moves a document from raw PDF to a queryable vector database.

Step	What students see	RAG concept demonstrated
1-2	Open Colab, install libraries	Tool setup
3	Install chromadb, pypdf, transformers	Dependency management
4	Upload PDF to Colab	Document ingestion
5	Extract and chunk pages	Chunking strategy
6	Create Chroma collection	Vector index creation
7	Embed and store chunks	Embedding + upsert
8	Query by question	Semantic retrieval
9	Inspect raw vector numbers	What an embedding IS
10	Student types their own question	Live RAG in action

STEP 01

Open Google Colab

1. **Open your browser and go to colab.research.google.com**
2. **Sign in with your Google account if prompted**
3. **Click the blue “New notebook” button in the top-left area**
4. **A new tab opens with one empty code cell that says *“Start coding or generate with AI”***

WHAT IS GOOGLE COLAB?

Google Colab is a free cloud-based Python environment that runs entirely in your browser. It gives you a Linux computer with Python pre-installed, internet access, and optionally a GPU — all for free, with no setup on your laptop.

Why it matters:

This is the tool most AI teams use for prototyping and demos. Running everything in Colab means every student sees identical behaviour regardless of their own laptop's operating system or Python installation.

STEP 02 Connect to a Runtime (GPU Optional)

1. **Click Runtime in the top menu bar**
2. **Click “Change runtime type” in the dropdown**
3. **Under Hardware accelerator, the default is None (CPU) — for this lab you can leave it as CPU; the demo runs fine**
4. **If you want to show students the GPU option: change to T4 GPU, click Save**
5. **You will see “Connected” appear in the top-right corner**

CPU vs GPU FOR THIS LAB

A GPU (Graphics Processing Unit) is a chip originally designed for video games that turns out to be perfect for the matrix multiplication that neural networks use. For large model training (like the Llama fine-tuning we discussed), a GPU is essential. For our Chroma demo today — embedding ~10 pages of text and doing a few similarity searches — the CPU is fast enough.

Why it matters:

Teaching moment: this is also a cost decision. GPU compute costs money. Good engineers only spin up a GPU when the workload actually needs it.

■■ Watch out:

Note: if you chose T4 GPU, Colab may take 30–60 seconds to allocate one. If none are available you’ll get a message saying so — just switch back to CPU and continue.

STEP 03 Install the Required Libraries

1. **Click into the first empty code cell**
2. **Paste the code below exactly**
3. **Press Shift + Enter to run (or click the ■ play button on the left of the cell)**
4. **Wait — this takes about 60 seconds. You'll see a progress bar with download speeds**
5. **When finished, you will see the cursor move to a new cell**

CODE TO PASTE:

```
!pip install chromadb pypdf sentence-transformers -q
```

WHAT ARE THESE THREE LIBRARIES?

chromadb: the vector database we are building today. pypdf: reads and extracts text from PDF files. sentence-transformers: converts text into numerical vectors (embeddings) so Chroma can do similarity search.

Why it matters:

These three libraries represent the three stages of a RAG pipeline: ingest (pypdf), embed (sentence-transformers), store and retrieve (chromadb). We are manually assembling what production systems like LangChain or LlamaIndex bundle together.

■■ Watch out:

You may see red text saying ERROR: pip's dependency resolver... This is a version conflict warning with other packages already on Colab — it does NOT affect our three libraries. Ignore it.

■ Expected output:

Confirm everything worked: click + Code, paste the 4 import lines below, and run. If you see the green tick, proceed.

STEP 04 Confirm Libraries Loaded

1. **Click + Code in the toolbar (top of screen, between File menu area and Run all)**
2. **A new empty cell appears below the previous one**
3. **Paste the code below and press Shift + Enter**

CODE TO PASTE:

```
import chromadb
import pypdf
from sentence_transformers import SentenceTransformer
print('■ All libraries loaded successfully! Ready to build the vector DB.')
```

WHAT IS + CODE?

In Colab, a notebook is made of cells. Each cell holds code you can run independently. The + Code button adds a new code cell. You can also hover between two cells to get the + Code / + Text bar that appears between them.

Why it matters:

Breaking work into cells is good practice — it lets you re-run just one section without re-running everything above. This is exactly how data scientists and AI engineers structure real experiment notebooks.

■ Expected output:

Expected output: ■ All libraries loaded successfully! Ready to build the vector DB.

STEP 05

Upload the PDF to Colab

1. **Look at the left sidebar — click the folder icon (Files panel)**
2. **Click the upload icon at the top of the Files panel (it looks like a page with an upward arrow)**
3. **In the file picker, navigate to and select AI-for-Teachers-an-Open-Textbook.pdf**
4. **Wait for the filename to appear in the Files panel list**
5. **The file is now stored in /content/ — Colab's working directory**

WHERE DOES THE FILE GO?

Colab gives you a temporary virtual hard drive at /content/. Anything you upload lives there for the duration of your session. If the session times out or is restarted, files are erased and need to be re-uploaded.

Why it matters:

This is the 'document ingestion' step. In a production RAG system this would be automated — files dropped into an S3 bucket, a SharePoint folder, or a database trigger kicks off the embedding pipeline automatically.

■■ Watch out:

The file is 12 MB — the upload usually takes 10–20 seconds on a typical broadband connection.

STEP 06 Extract and Chunk the PDF

1. **Click + Code to add a new cell**
2. **Paste the code below**
3. **Press Shift + Enter**
4. **You should see output listing each page number and character count**

CODE TO PASTE:

```
from pypdf import PdfReader

reader = PdfReader('AI-for-Teachers-an-Open-Textbook.pdf')

# Extract pages 39-48: Chapter 9, AI Speak: Machine Learning
chunks = []
for i, page in enumerate(reader.pages[38:48], start=39):
text = page.extract_text()
if text and len(text.strip()) > 50:
chunks.append({
'id': f'page_{i}',
'text': text.strip(),
'page': i
})

print(f'■ Extracted {len(chunks)} chunks')
for c in chunks:
print(f' Page {c["page"]}: {len(c["text"])} characters')
```

WHAT IS CHUNKING AND WHY PAGES?

Chunking is splitting a long document into smaller pieces that fit inside an embedding model's context window and make sense as standalone units. Here we use one page as one chunk. In production you'd tune this — some teams chunk by paragraph, by sentence, or by a fixed token count with overlap between adjacent chunks.

Why it matters:

This is exactly what your students did manually in the RAG Lab exercise — picking relevant sections rather than pasting the whole document. Chunking strategy is one of the most important decisions in a RAG system: too large and retrieval is imprecise, too small and the retrieved chunk lacks enough context.

■ Expected output:

Expected output: ■ Extracted 10 chunks, followed by page numbers and character counts.

STEP 07 Create the Chroma Vector Database

1. **Click + Code to add a new cell**
2. **Paste the code below**
3. **Press Shift + Enter**

CODE TO PASTE:

```
import chromadb

# Create a local in-memory Chroma client
client = chromadb.Client()

# Create a collection (like a table, but for vectors)
collection = client.create_collection(
name='ai_textbook',
metadata={'hnsw:space': 'cosine'}
)

print('■ Chroma vector database created')
print(f'Collection name: {collection.name}')
print(f'Using cosine similarity to measure closeness between vectors')
```

WHAT IS A CHROMA COLLECTION?

A Chroma Client is the database engine running in memory. A Collection is like a table inside that database — but instead of rows of text, it stores vectors (lists of numbers). The metric='cosine' setting tells Chroma to measure similarity using cosine distance: two vectors pointing in the same direction have cosine similarity of 1.0 (identical meaning), opposite directions give 0.0 (unrelated meaning).

Why it matters:

This is the 'index creation' step. Compare to what search engines do: Google's index maps keywords to pages. Our vector index maps semantic meaning to chunks. The difference: keyword search finds exact word matches; vector search finds meaning matches, even when different words are used.

■ Expected output:

Expected output: ■ Chroma vector database created, Collection name: ai_textbook

STEP 08

Embed the Chunks and Store in Chroma

1. **Click + Code to add a new cell**
2. **Paste the code below**
3. **Press Shift + Enter**
4. **This step takes 15–30 seconds as it downloads the embedding model on first run**

CODE TO PASTE:

```
# Add all chunks to Chroma
# Chroma will automatically embed them using a built-in model
collection.add(
ids=[c['id'] for c in chunks],
documents=[c['text'] for c in chunks],
metadatas=[{'page': c['page']} for c in chunks]
)
print(f'■ {collection.count()} chunks stored in the vector database')
print('Each chunk has been converted to a vector of 384 numbers')
print('Chroma can now find the most SIMILAR chunk to any question')
```

WHAT JUST HAPPENED UNDER THE HOOD?

When you called collection.add(), Chroma took each text chunk, passed it through a sentence transformer model (all-MiniLM-L6-v2 by default), and got back a vector of 384 numbers representing the semantic meaning of that text. Those 384 numbers are stored in an in-memory HNSW (Hierarchical Navigable Small World) index — a data structure optimised for fast nearest-neighbour search in high-dimensional space.

Why it matters:

This is the most important step — the moment text becomes searchable by meaning, not by keyword. The 384 numbers don't mean anything individually; their pattern relative to each other encodes meaning. Similar sentences will produce similar patterns of numbers.

■ Expected output:

Expected output: ■ 10 chunks stored in the vector database

STEP 09 Query the Database with a Question

1. **Click + Code to add a new cell**
2. **Paste the code below**
3. **Press Shift + Enter**
4. **Read the output aloud to students — pause on the page number returned**

CODE TO PASTE:

```
question = 'What is machine learning and how does it work?'

results = collection.query(
query_texts=[question],
n_results=2
)

print(f'■ Question: {question}')
print('=' * 60)

for i, (doc, meta) in enumerate(zip(
results['documents'][0],
results['metadatas'][0]
)):
print(f'\n■ Result {i+1} — Page {meta["page"]}')
print('-' * 40)
print(doc[:500])
print('...')
```

WHAT HAPPENED DURING THIS QUERY?

Chroma took your question, converted it into a 384-number vector using the same embedding model, then searched the index for the stored chunks whose vectors are most similar (closest in cosine distance) to the question vector. The top 2 results are returned with their metadata (page number). No keyword matching happened — Chroma found relevant text because the meaning matched, not the words.

Why it matters:

Ask students: 'Did the word machine learning have to appear on that exact page?' Try a paraphrased question next: 'How do computers learn from data?' — you should get the same result page back. That’s the power of semantic retrieval over keyword search.

■ Expected output:

Expected output: two result blocks each showing a page number and the first 500 characters of that page’s text.

STEP 10 Inspect What a Vector Actually Looks Like

1. **Click + Code to add a new cell**
2. **Paste the code below**
3. **Press Shift + Enter**
4. **Point to the array of numbers on screen and say: *“This is what the database actually stores.”***

CODE TO PASTE:

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')

question = 'What is machine learning?'
vector = model.encode(question)

print(f'Your question as a vector ({len(vector)} numbers):')
print(np.round(vector[:12], 4), '...')
print()
print('This is what Chroma stores for each chunk and each query.')
print('Similar meaning = vectors pointing in similar direction.')
print('Chroma finds closest vectors using cosine similarity.')
```

MAKING EMBEDDINGS VISIBLE

This cell shows students the actual numbers produced by the embedding model for a short question. Each of the 384 numbers is a floating-point value between roughly -1 and +1. The full pattern of 384 numbers encodes the semantic meaning of the sentence. A slightly different sentence produces a slightly different pattern; a completely unrelated sentence produces a completely different pattern.

Why it matters:

Most students have heard the word 'embedding' without ever seeing one. Showing the actual array — and pointing out it’s 384 meaningless-looking decimals — makes the abstraction concrete and memorable. Ask: 'Could you reverse-engineer the original sentence from these numbers?' (No — the encoding is lossy by design.)

■ Expected output:

Expected output: an array like [-0.0231 0.0412 -0.1023 0.0875 ...] followed by the three explanation lines.

STEP 11 Live Student Question — Hands on Keyboard

1. **Click + Code to add a new cell**
2. **Paste the code below**
3. **Hand the keyboard (or mouse) to a student**
4. **Ask them to change the question string to anything they are curious about**
5. **Have them press Shift + Enter and read the result aloud**
6. **Repeat with 2–3 students if time allows**

CODE TO PASTE:

```
# Student: change the question below to anything you want to ask the textbook!
question = 'How does AI affect student privacy?' # <-- change this

results = collection.query(query_texts=[question], n_results=1)

print(f'Question: {question}')
print(f'\nBest matching chunk from the textbook:')
print('-' * 50)
print(results['documents'][0][0][:600])
print(f'\nFrom page: {results["metadatas"][0][0]["page"]}')
```

WHY THIS STEP MATTERS

This is the moment where the abstract concept becomes tangible for students. The textbook has never been keyword-indexed for their question — the system finds the right page purely based on semantic similarity between their question vector and the stored chunk vectors.

Why it matters:

Challenge students to try a question where none of the exact words appear in the textbook and see if retrieval still works. Try questions about topics covered in chapters we did NOT embed (pages outside 39-48) — it will fail to find a good match, which teaches retrieval coverage and the importance of indexing your full document corpus.

■■ Watch out:

If a student asks something completely unrelated to AI or education, the result will look like a bad match — that’s fine and worth discussing: RAG systems only know what you’ve put into them.

LAB SUMMARY

What Was Achieved

In this lab you built a complete, working Retrieval-Augmented Generation (RAG) pipeline from scratch using nothing but free tools running in a browser. Here is exactly what was constructed, step by step:

✓ Document Ingestion

Loaded a 250-page academic PDF into a Python environment using pypdf — the same technique used in every production document pipeline.

✓ Chunking

Split the document into 10 page-level chunks, each representing an independent unit of retrievable knowledge. Chunk strategy directly determines retrieval quality.

✓ Embedding

Converted raw text into 384-dimensional numerical vectors using a sentence transformer model. Each vector encodes the semantic meaning of its chunk.

✓ Vector Index Creation

Created a Chroma collection using cosine similarity as the distance metric — the same metric used by Pinecone, Weaviate, Qdrant, and every other production vector database.

✓ Semantic Retrieval

Queried the database with natural-language questions and retrieved the most semantically similar chunks — not by keyword match, by meaning.

✓ Hands-on Exploration

Students experienced RAG first-hand by submitting their own questions and seeing which page the system retrieved, and why.

Key Concepts Demonstrated

Concept	Plain English Definition	Shown in Step
Chunking	Splitting a document into retrievable pieces	Step 6
Embedding	Converting text into a list of numbers (vector) that encodes meaning	Steps 8, 10

Vector Database	A database optimised to store and search by vector similarity	Step 7
Cosine Similarity	A measure of how 'close' two vectors are in meaning (1.0 = identical)	Step 7
Semantic Search	Finding relevant content by meaning, not exact keyword match	Steps 9, 11
RAG Pipeline	The full loop: chunk → embed → store → retrieve → answer	All steps

How this connects to Session 6 (Fine-Tuning vs RAG)

This lab demonstrates the RETRIEVAL side of your slide 4 comparison. RAG retrieves relevant text at query time and injects it into the prompt — which is exactly what Chroma just did. Fine-tuning, by contrast, bakes knowledge into the model weights during training — no retrieval step needed at inference, but also no ability to cite a source or stay current without retraining. Both approaches solve the 'LLM doesn’t know your private data' problem — they just do it at different points in the pipeline.

What Students Can Do Next

- Add more chapters from the PDF (change the page range in Step 6) and see how retrieval quality changes
- Connect ChromaDB retrieval to ChatGPT via the API (Session 8 lab) to build a full chat-with-your-document system
- Replace Chroma with Pinecone for a persistent, cloud-hosted vector database that survives session resets
- Experiment with chunk size — try sentence-level chunks vs paragraph-level and compare retrieval quality
- Add metadata filtering: only retrieve chunks from specific page ranges or chapters